

Trabajo Práctico 3

Materia: Programación III

Alumno: Paz Matias Leonel

Tema: Aplicación de delivery, tiene posibilidad de pedir comida ya preparadas o otros articulos

Introducción

Para realizar este trabajo desarrollé una aplicación de delivery en Java y utilicé seis patrones de diseño, La idea era aplicar cada patrón a una situación diferente dentro de la aplicación.

Los patrones utilizados fueron:

- Singleton
- Factory Method
- Abstract Factory
- Adapter
- Decorator
- Facade

1. Singleton

Escenario

En la aplicación necesitamos tener una configuración general con datos como la moneda que en esta app usamos pesos argentinos obvio, pero podemos utilizarlo para otros tipos de moneda y también costo base del envío.

Justificación

Utilicé Singleton porque necesito que exista una sola configuración en todo el sistema. Para eso, ConfiguracionDelivery tiene un constructor privado y un método getInstance() que devuelve siempre la misma instancia.

Problemas encontrados

Al iniciar tuve algunos problemas con organizar los paquetes y no podía iniciar la instancias, luego fui separando bien uno por uno y utilice el getInstance() para configurarlo bien.

2. Factory Method

Escenario

La aplicación puede tener diferentes tipos de pedidos como lo mencione antes, por ejemplo pedidos de comida y distintos tipos de artículos de supermercado.

Justificación

Utilicé Factory Method para que la creación de los pedidos se haga desde PedidoFactory, sin tener que crear directamente cada tipo de pedido desde Main.

De esta forma, dependiendo del tipo indicado, la fábrica crea PedidoComida o PedidoSupermercado.

Problemas encontrados

El problema principal que tuve fue entender la diferencia entre la fábrica y los objetos que crea. Lo solucioné haciendo que ambos tipos de pedido implementen la interfaz Pedido.

3. Abstract Factory

Escenario

La aplicación tiene diferentes tipos de familias de productos. En este caso tenemos una familia tradicional y otra premium, Cada familia tiene un menú y un método de pago.

Justificación

Utilicé Abstract Factory porque permite crear una familia de objetos relacionados.

Por ejemplo:

Tradicional:

- MenuTradicional
- PagoTradicional

Premium:

- MenuPremium
- PagoPremium

Las fábricas DeliveryFactoryTradicional y DeliveryFactoryPremium se encargan de crear cada familia.

Problemas encontrados

Me costó diferenciar Abstract Factory de Factory Method. La diferencia que entendí es que Factory Method crea un tipo de producto, mientras que Abstract Factory crea una familia de productos relacionados

4. Adapter

Escenario

La aplicación necesita procesar pagos, pero tenemos una pasarela externa que utiliza métodos diferentes a los que utiliza nuestro sistema.

Justificación

Utilicé Adapter para conectar PasarelaExterna con la interfaz ProcesadorPago.

La clase PasarelaPagoAdapter funciona como intermediaria y adapta el método de la pasarela externa al método que necesita nuestra aplicación.

Problemas encontrados:El problema principal fue entender qué clase debía adaptar a cuál. Lo solucioné haciendo que el Adapter implemente la interfaz que utiliza nuestra aplicación y que internamente utilice la pasarela externa

5. Decorator

Escenario

Un pedido básico puede tener servicios adicionales, como envío express y seguro.

Justificación

Utilicé Decorator porque permite agregar funcionalidades al pedido sin modificar la clase original.

Primero tenemos:

PedidoBasico

Después podemos agregar:

EnvioExpress

y luego:

SeguroPedido

Esto permite combinar diferentes servicios.

Problemas encontrados

Al principio me costó entender cómo se podían agregar varios decoradores. Finalmente entendí que cada uno envuelve al objeto anterior y agrega su propio comportamiento.

6. Facade

Escenario

Para realizar un pedido existen diferentes operaciones internas, como validar y procesar el pedido.

Justificación

Utilicé Facade para simplificar estas operaciones.

La clase SistemaPedidos funciona como una fachada y se encarga de utilizar:

- ValidadorPedido
- ProcesadorPedido

Desde Main solamente tenemos que llamar:

```
sistema.realizarPedido();
```

Problemas encontrados

La dificultad fue entender qué clases debían quedar detrás de la fachada. Lo solucioné dejando las operaciones internas en ValidadorPedido y ProcesadorPedido, y utilizando SistemaPedidos como punto de acceso.